Design a URL Shortening Service

- URL Shortening is used to create shorter aliases for long URLS.
  So whenever user hits these shorter aliases, they are redirected to original URL.

eg :- www.neet.ac.in/qmtqapqstemx

Long URL → wampinqki/mops37tqqr

www.tinyurl.com/pqxyz10t

8 length

eg

Functional Requirement :-

1. Whenever you are given a URL, service should generate a shorter and unique aliases.

2. Whenever the users access a short URL, service should redirect them to the original long link.

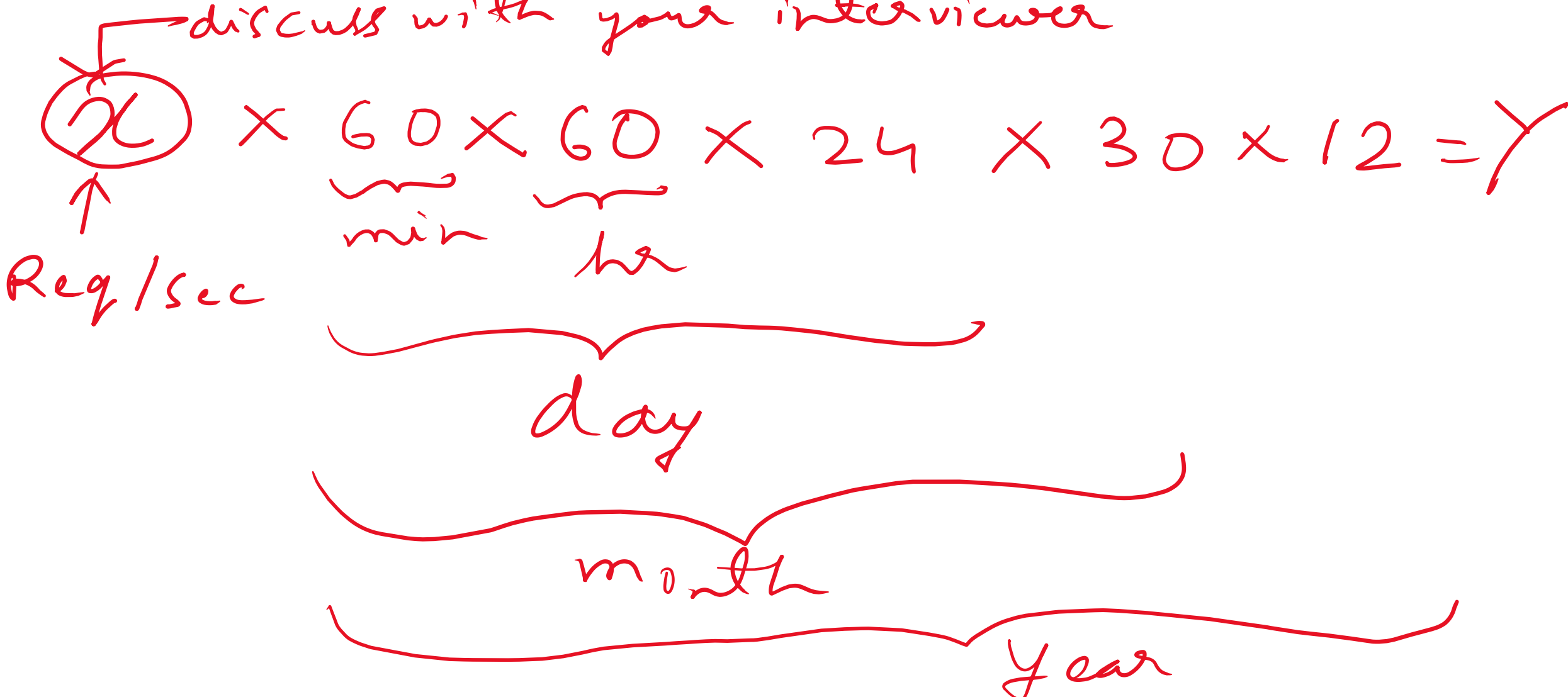
① Non Functional Requirement :-

1. Scalable

2. Available

3. Low Latency

⊙ Capacity Estimation (Traffic) :-



- Write means you are provided with LONG URL and you need to convert it into short URL.

- READ means that you are hitting a short URL and that needs redirected to long URL.

- [a-z] [A-Z] [0-9]

26 + 26 + 10 = 62

shortURL alias of

1 length = 62

^

2 length = 62 × 62

3 length = 62^3

⋮

8 length = 62^8

- Long URL provided to you

1. Compute hash of the URL provided to you.

hash[key] = value;

- Issue :- If multiple users are entering same URL, they could get same shortened URL, which is not unique.

① Possible solution to above Issue :-

1. Append an increasing sequence of number to the input long URL, so that it is unique.

( ) + ① ⇒ unique

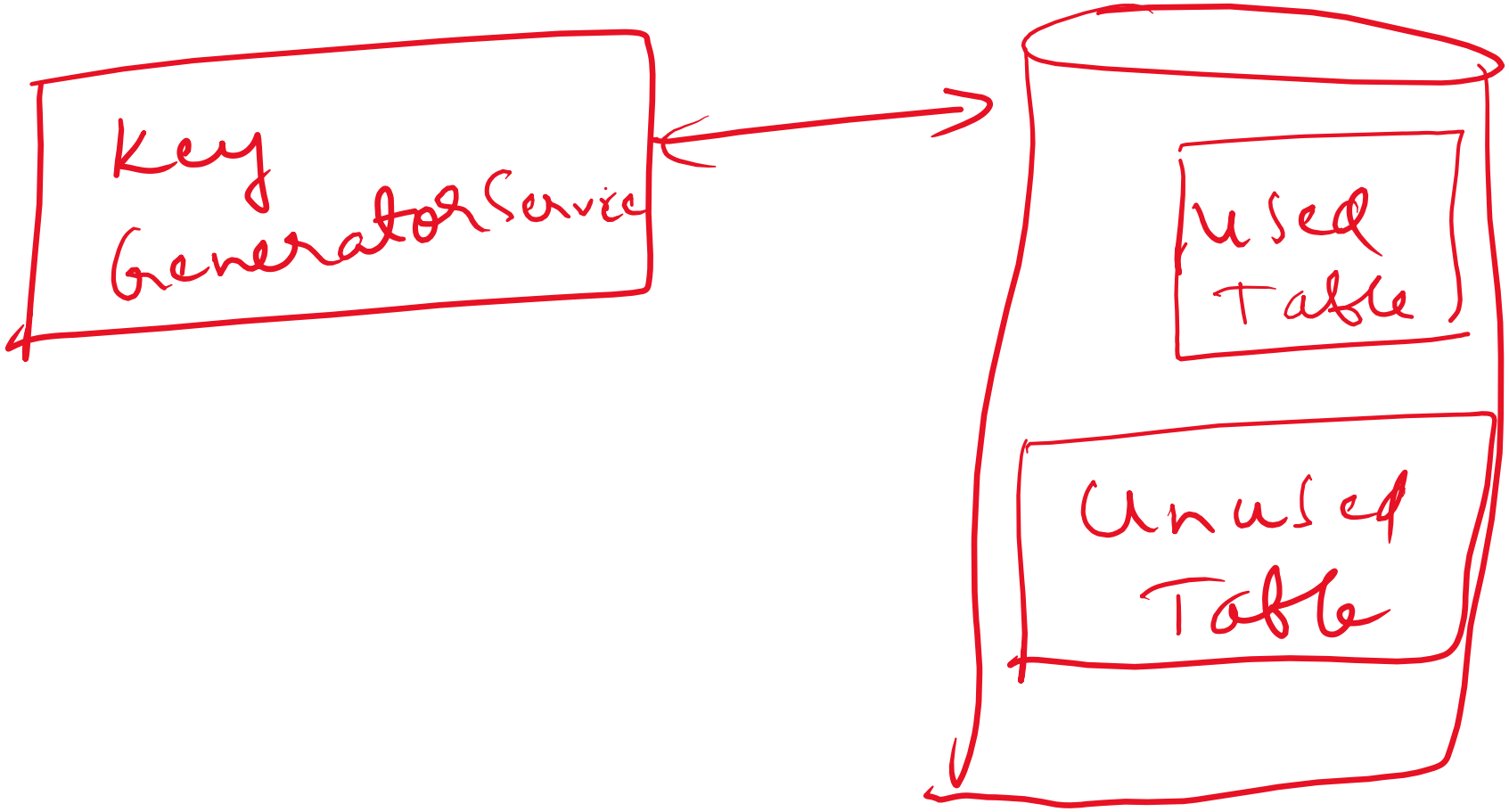
( ) + ② ⇒ unique.

2. Append User ID

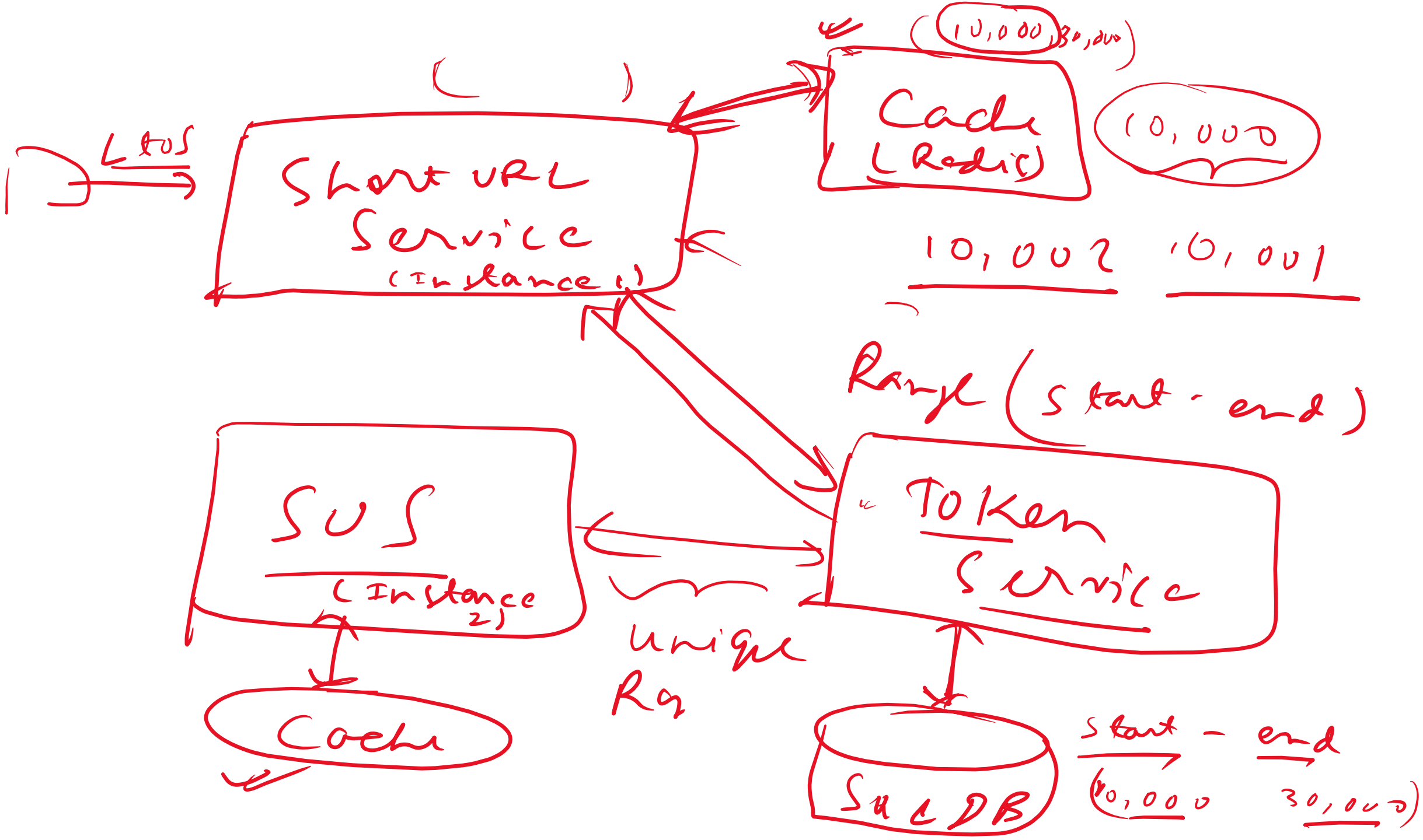
Hash (long URL + User ID) → Unique value

Key

- Use Key Generator Service :- It would generate a random 8 letter string beforehand and store them in a DB and whenever we wanna shorten a URL, we will just take one of the keys and use it.


Key Generator Service
Used Table
Unused Table

⊙ Token Service :- It is going to store range of numbers (1000 ~ 30,000) and whenever any request comes up, it will assign that specific range to the instance making the request and once it's assigned, remove it from Token Service.


Lbls
Short URL Service (Instance 1)
Cache (Redis)
(10,000 30,000)
(0,000
10,002 10,001
Range (start - end)
SUS (Instance 2)
Token Service
Unique Rq
Cache
SqlDB
start - end
(0,000 30,000)